CORPORATE TRAINING PROGRAM · 20 HOURS · 5 HALF-DAYS

AI-Enabled Project & Delivery Management



Generative AI, predictive analytics and automation for experienced Project and Delivery Managers — with the Software Factory, an agent-driven delivery pipeline, as the running case.

GenAI People · Jerry Kurian · genaipeople.com

Prepared for Gynosis Information Technology Pvt. Ltd. · Client: UST

WHY THIS PROGRAM

The Project Manager's job is changing — twice

First shift: AI in the PM's own work. Charters, estimates, risk registers, status reports and stakeholder updates are now drafted in minutes. The skill moves from writing to reviewing.

Second shift: AI doing the delivery. Agents now spec, plan, design, implement, review and test. The PM stops coordinating hand-offs and starts holding gates.

What does not change. Scope, trade-offs, accountability, relationships, and the decision to ship. These get more valuable, not less.

The program teaches both halves, with hands-on labs every day and a real agent-driven pipeline on Days 4 and 5.



Assistant

Drafts for you. You paste context in, you copy the output out. Day 1.



Copilot

Works inside your tools — tracker, chat, spreadsheet, IDE. Days 2 and 3.



Agent

Does the work end to end. You approve at gates. Days 4 and 5.

PROGRAM AT A GLANCE

Five half-days, one running case

1 — 4 h

GenAI for the PM

Foundations · forecasting · risk

- Intro to GenAI in PM
- Forecasting & risk
- Lab: plan + forecast

2 — 4 h

Automation & reporting

Workflows · status · comms

- AI automation
- AI reporting
- Lab: automated status

3 — 4 h

Tools & integration

Landscape · integration · monitoring

- Modern PM tools
- Collaboration tools
- Monitoring
- Lab: tool integration

4 — 4 h

AI-driven delivery

The Software Factory operating model

- Future of PM
- Delivery best practices
- Managing Gen AI projects
- Lab: run an epic

5 — 4 h

Capstone & certification

Case study · capstone · 30-60-90

- Case study
- Capstone (teams)
- Conclusion & certificate

Every day: three 50-minute teaching-and-demo blocks, one 60–90-minute lab, a closing round. **Every module** ties back to the Software Factory so the concepts are seen running, not just described.

AUDIENCE AND FORMAT

Built for senior delivery practitioners

20 h
five half-days of 4–4.5 hours, online, recorded

8
hands-on labs, one every half-day or more

1
capstone: a full AI-enabled delivery plan, presented

12–20
participants per batch; capstone teams of 3–4

Who it is for

- Career Grade 4–6 Project and Delivery Managers
- 12–18 years total experience, 5–6 in project / delivery management
- Running agile or hybrid delivery with Jira, Azure DevOps or GitHub
- No coding required — labs are browser, spreadsheet and guided templates

How it is taught

- Concept → live demo → lab on participants' own data or a sandbox
- Sandbox GitHub organisation with the Software Factory pre-installed
- Participants sit in the approver seats on Days 4–5 and open real gates
- Assessment: capstone 60% + scenario quiz 40% → certificate of completion

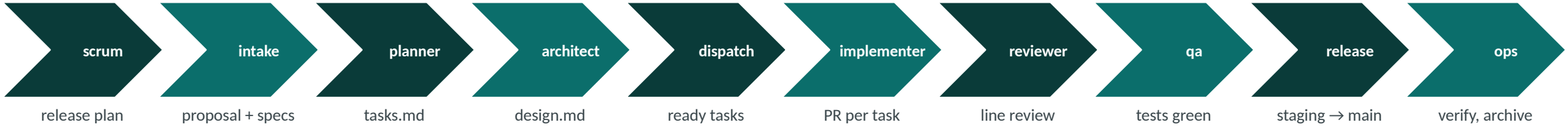
BUSINESS OUTCOMES

Every expected business outcome has a module and a proof

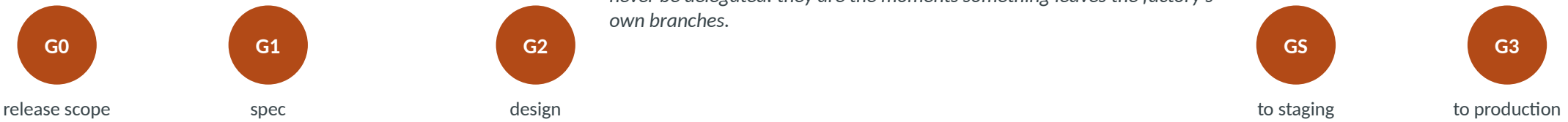
Expected business outcome	Where it is built	Evidence at the end
Improved project delivery predictability	Day 1 forecasting · Day 4 small batches, staging first, release gating	P50 / P85 forecast on real data; sequenced release plan
Faster and more accurate project reporting	Day 2 reporting from the system of record · Day 3 monitoring	A working automated status report posted to Teams / Slack
Reduced risk exposure through AI-driven forecasting	Day 1 risk mining and early-warning signals · Day 4 independent review, gates	AI-mined risk register with exposure and signals
Increased productivity through workflow automation	Day 2 automation · Day 3 integration · Day 4 agents doing the work	Automation built in the lab; an epic run end-to-end by agents
Better resource utilisation and governance	Day 3 flow metrics · Day 4 gates, approvers, guardrails, model routing	Governance design in the capstone
Readiness to manage AI-driven business initiatives	Day 4 managing Gen AI projects · Day 5 capstone	Capstone delivery plan and personal 30-60-90 plan

The running case: the Software Factory

A plain GitHub issue goes in. A specced, planned, designed, implemented, reviewed, tested and released change comes out — with human approval gates, and no ability for an agent to put anything on main.



Human gates



Everything between the gates runs unattended. Only GS and G3 can never be delegated: they are the moments something leaves the factory's own branches.

Why it is the running case: the client's six outcomes — predictability, reporting, risk, automation, governance, AI readiness — are exactly what a factory-style pipeline forces you to design for. Participants see every concept of the program running on a real repository, and sit in the approver seats themselves.

Two foundations, one rule

GitHub is the state machine

Milestones are releases; issues, sub-issues and factory:* labels encode where every piece of work is. A label change is what wakes an agent.

For the PM: reporting becomes a query, not a meeting. Flow metrics come for free. Any human can take over any stage by moving the label.

OpenSpec is the content

What is being built — proposal, WHEN/THEN scenarios, technical design, task checklist — lives in the repo as plain Markdown, versioned with the code.

For the PM: the artifacts you review at a gate are short documents, not transcripts. Requirements, plan and design are one folder, one history.

Issues carry state plus a link — never spec content

Agents read requirements from the change folder, never from the issue body. That is what stops the two sources of truth from drifting.

For the PM: the single most common cause of delivery failure — the ticket says one thing, the document another — is designed out.

DAY 1 AGENDA · 4 HOURS

Day 1 — Generative AI for the project manager

0:00 – 1:00	**Introduction to Generative AI in PM** LLMs for PMs · assistant / copilot / agent · prompting patterns
1:00 – 1:10	**Break**
1:10 – 2:25	**AI-driven forecasting and risk management** Throughput, Monte Carlo, risk mining, early-warning signals
2:25 – 2:35	**Break**
2:35 – 3:50	**Hands-on: from brief to plan; forecast** Lab 1A charter / WBS / risks · Lab 1B Monte Carlo
3:50 – 4:00	**Closing round**

Client ToC modules covered

- Introduction to Generative AI in Project Management
- AI-Driven Project Forecasting and Risk Management
- Hands-On Exercise

Software Factory thread

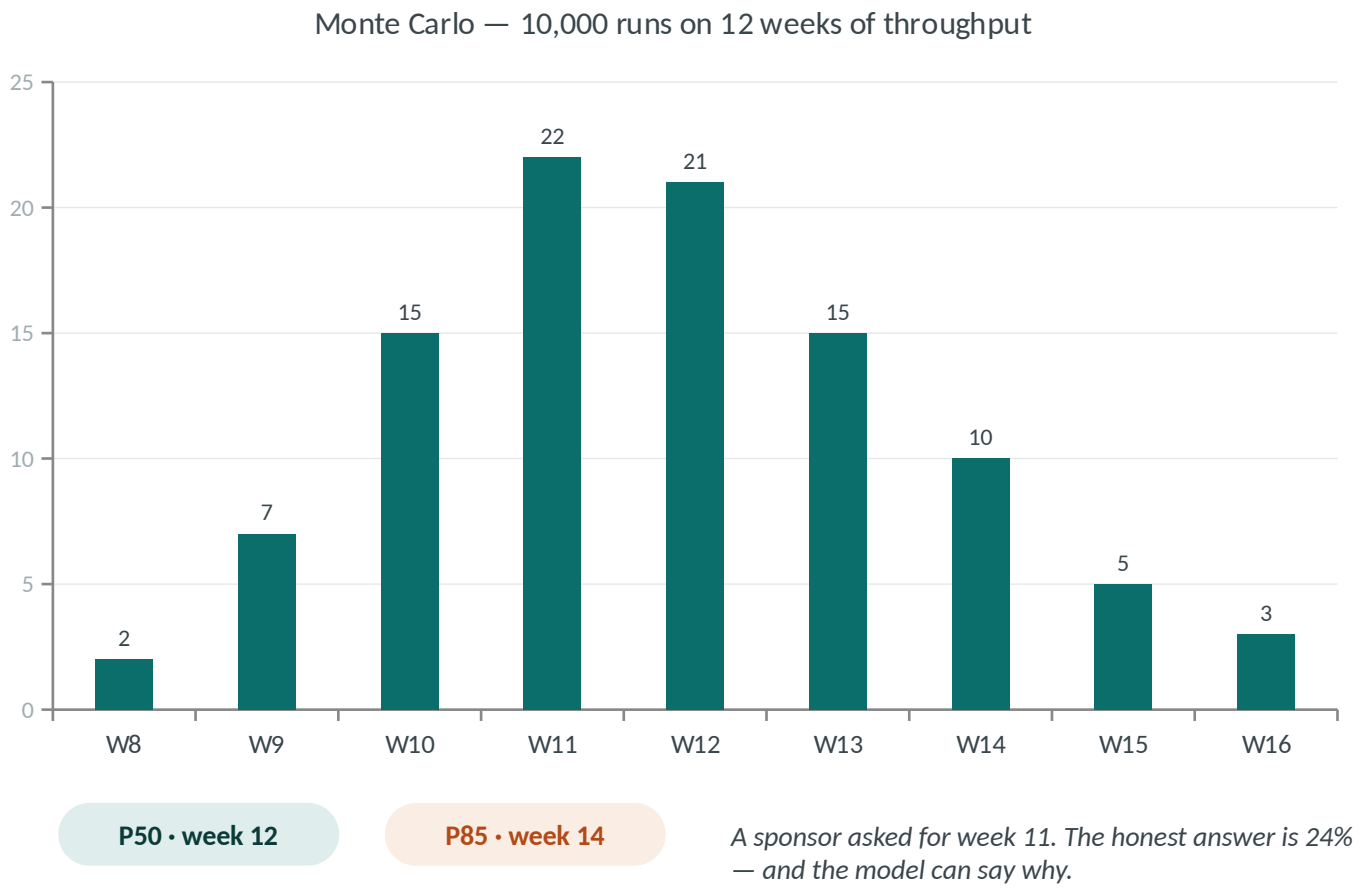
- The Intake Analyst: an agent that stops and asks (factory:blocked) instead of inventing requirements
- The Scrum Master: release scope, sequencing, risks and a GO / NO-GO on the whole milestone before anything starts

Where Generative AI fits in the PM lifecycle

Initiate	Plan	Execute	Monitor	Close
• Charter and business case drafts • Stakeholder map • Assumption and question lists	• WBS, RACI, schedule skeleton • Estimate ranges with rationale • Requirements → acceptance scenarios	• Meeting → decisions and actions • Change-request summaries • Agents implementing tasks	• Status from the system of record • Forecasts and risk signals • Attention queue: what needs me	• Lessons learned from the record • Archive specs, decisions • Benefits tracking narrative
Assistant	**Assistant → Agent**	**Copilot → Agent**	**Copilot**	**Assistant**

Where it does not fit: deciding scope, negotiating trade-offs, owning the relationship, and saying yes or no at a gate. The prompting rule that matters most in delivery: ask the model for its questions and assumptions before its answer.

Forecasting: from a date to a probability



A sponsor asked for week 11. The honest answer is 24% — and the model can say why.

Count things, not points. Throughput and cycle time forecast better than estimates, and need no re-estimation.

Simulate, then explain. Monte Carlo in Excel or a guided notebook gives P50 / P85; the LLM turns it into the sponsor conversation.

Stress-test the inputs. Ask the model for the scenarios that would move the P85 by two weeks — scope churn, WIP, blocked ageing.

Factory tie-in. Because every stage is a label with a timestamp, the factory produces per-stage cycle-time data by itself. Forecasting an epic is a query.

AI-driven risk management: mine it, classify it, watch for it

Three moves

Mine. Risks hide in tickets, retros, incident logs and meeting transcripts. An LLM reads all of it and proposes a register with sources cited.

Classify. Probability × impact is still the model; the AI drafts, the PM decides. Pre-mortems generate the risks nobody wants to say aloud.

Watch. Early-warning signals from the system of record: blocked ageing, review-queue depth, re-open rate, scope churn, cycle-time drift. Automate the watch; escalate on threshold.

Factory tie-in. The Scrum Master (right) does the first two moves on a whole release before anything starts; the state labels give you the third for free.

`factory:release-ready`

Release plan — release(3): Policy servicing Q4

Posted by the Scrum Master role after reading every issue in the milestone

Issue	Intent	Seq	Concern
#41	Renewal reminders via WhatsApp	1	—
#42	Premium recalculation API	2	schema first
#43	Agent dashboard	3	depends on #42
#44	Bulk import	—	oversized: split
#45	Fix date format	—	fast-track

Risks 1. #42 has no owner for the rate table (ask: actuarial). 2. #43 and an in-flight change overlap on the dashboard module.
Verdict **GO once #44 is split and #45 leaves the milestone.**

AI release planning on the set, not the item: duplicates, sequencing, size and under-specification are caught before a single agent starts. Gate G0 is the human's — or, by config, the agent's own GO.

Lab 1: from brief to plan, then from data to a date

1A · Brief → charter, WBS, RACI, risks

- Take the supplied one-page product brief
- Ask the model for its questions and assumptions first
- Generate charter, WBS, RACI and a risk register
- Run a structured review pass: what is wrong, what is missing, what is invented
- Mark up corrections side by side with the AI draft

1B · Throughput → P50 / P85

- Load the 12-week throughput dataset (or your own tracker export)
- Build the Monte Carlo in Excel or the guided notebook
- Read the P50 and P85 dates; try a WIP-limit scenario
- Have the LLM draft the sponsor explanation and the two scenarios that move the date

Deliverable A marked-up AI-drafted plan with the PM's corrections, and a probabilistic forecast with a sponsor-ready explanation.

Closing round

What changed in the plan once you asked the model for its questions first?

Which date would you have committed to before today, and which will you commit to now?

DAY 2 AGENDA · 4 HOURS

Day 2 — AI for project automation and reporting

0:00 – 1:00	**Using AI for project automation**	Triggers, workflow tools, the LLM step, human-in-the-loop design
1:00 – 1:10	**Break**	
1:10 – 2:10	**AI-driven reporting and stakeholder communication**	Reports from the system of record · meeting intelligence · citations
2:10 – 2:20	**Break**	
2:20 – 3:50	**Hands-on: automated status; meeting to decisions**	Lab 2A weekly status to Teams / Slack · Lab 2B decision log
3:50 – 4:00	**Closing round**	

Client ToC modules covered

- Using AI for Project Automation and Reporting
- Hands-On Exercise

Software Factory thread

- The label state machine: one state per issue, labels trigger agents, agent comments never self-trigger
- Every run ends by saying what is expected of the next actor
- The Factory Console: a lens, never a second source of truth — attention queue, decision log, cited Q&A

The automation stack a PM can own

Trigger
Webhook, schedule, label change, new message

Workflow
Power Automate, n8n, Zapier, GitHub Actions

LLM step
Summarise, classify, draft, extract, route

Destination
Teams, Slack, email, Confluence, the tracker

Human review
Approve, edit, or let it through by policy

May do alone

- Post a status draft to a channel
- Label and route new intake
- Remind owners of ageing items
- Extract actions from a transcript

Must draft for approval

- Client-facing communications
- Scope or date changes
- Escalations to a sponsor
- Anything that names a person's performance

Must never touch

- Merging to production
- Approving a gate on a human's behalf
- Deleting records
- Sending data outside the client's policy

Report from the system of record, not from people

`factory:backlog` → `factory:intake` → `factory:spec-ready` → `factory:spec-approved` → `factory:planned` → `factory:design-ready`

`factory:design-approved` → `factory:ready` → `factory:in-review` → `factory:in-test` → `factory:ready-to-ship`

`factory:in-staging` → `factory:deployed`

Exactly one state label per issue. Orange states wait on a human. Every transition has a timestamp, an actor and a comment — so status, cycle time, WIP and blocked ageing are queries.

Inputs

- Issues, sub-issues, labels, milestones
- Pull requests and reviews
- CI runs and deployments
- Calendar and meeting transcripts
- Never: a status meeting

The LLM is the narrator, not the source

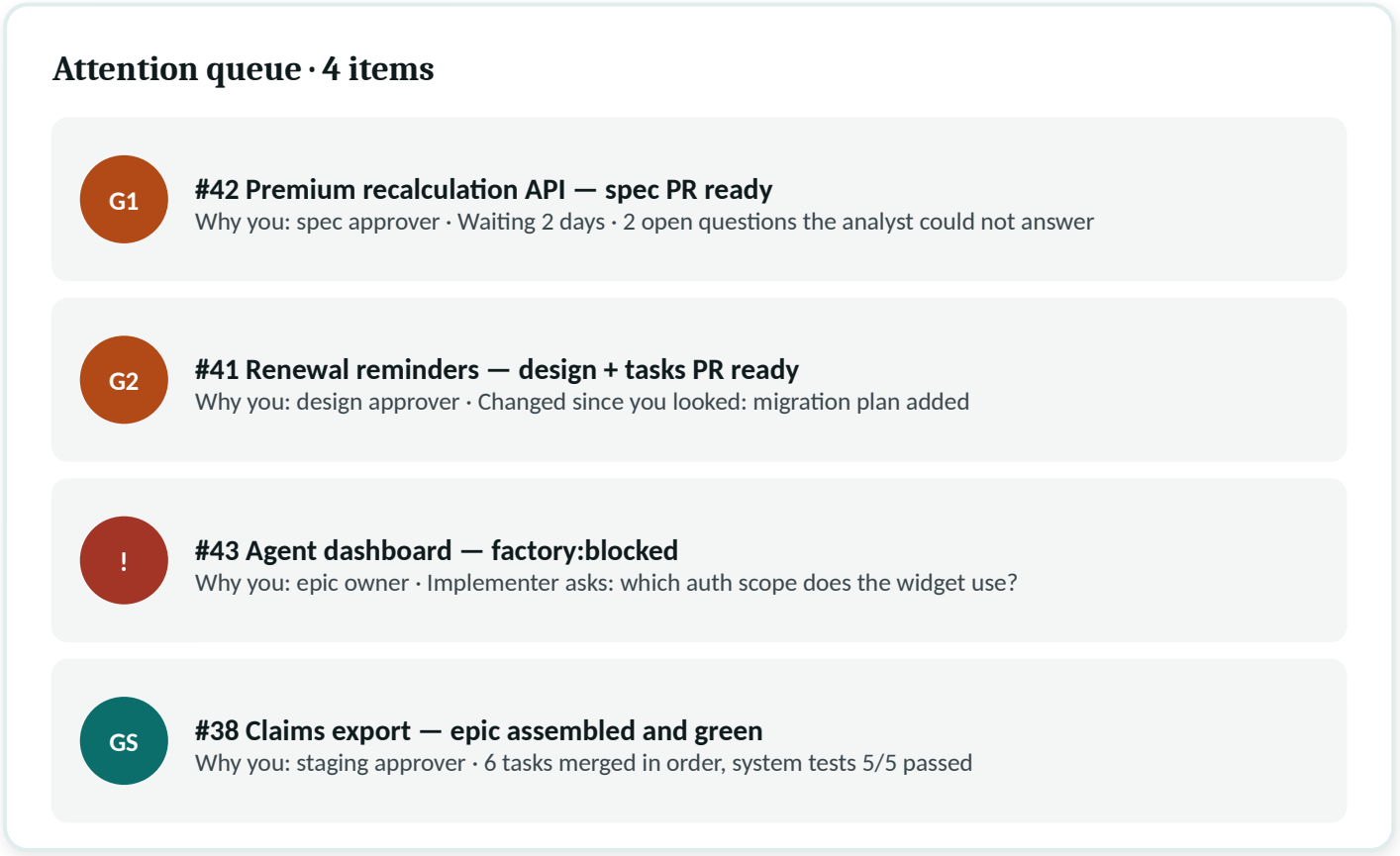
- One dataset, three audiences: sponsor, steering committee, team
- Each claim cites the object it came from
- As-of timestamps on every number
- A review step that costs less than the mistake

Meeting intelligence

- Transcript → decisions, actions, new risks
- Decision log kept current automatically
- In the factory: decisions are posted as ordinary GitHub comments with hidden markers — readable anywhere, indexed by the Console

What an AI-era PMO dashboard looks like: the attention queue

"What needs me, and why" — a lens over GitHub, never a second source of truth. Drop its tables, replay the events, and it is whole again.



Attention, not activity. The queue lists decisions waiting on you, each with the reason it is yours and how long it has waited.

What changed since you looked. As-proposed ⇄ as-approved diffs of the spec and design, so a re-review is minutes.

Decision log. Every gate approval is a recorded decision; every recorded decision is searchable.

Ask the Factory. Cited Q&A that answers only from the epic's own artifacts, threads, PRs and runs — with a token budget per organisation.

Lesson for any dashboard: show the person what needs them, cite the source object, and never become the place the truth lives.

DAY 2 · HANDS-ON EXERCISE

Lab 2: a status report nobody has to write

2A · Automated weekly status

- Connect the sandbox repo (or your tracker export) to the workflow tool
- Pull open issues, PRs, blocked items and deployments
- LLM step: sponsor-grade and team-grade versions from the same data, every claim cited
- Post both to a Teams / Slack channel; add a human-review branch for the sponsor version

2B · Meeting → decisions

- Take a supplied 40-minute transcript
- Extract decisions, actions and new risks, each with a timestamp
- Post the decisions as structured comments on the epic
- Compare with the human minutes: what did each miss?

Deliverable A working automated status pipeline posting to a channel, and a decision log generated from a transcript with citations.

Closing round

Which of your current reports could disappear next month?

What would you need to trust before the sponsor version goes out unreviewed?

DAY 3 AGENDA · 4 HOURS

Day 3 — Technology and tools for modern project management

0:00 – 0:50	**Overview of modern PM tools** Jira, ADO, GitHub, Asana, Linear, Notion, MS Project — and where the AI is
0:50 – 1:40	**Integrating collaboration and communication tools** Webhooks, APIs, bots, MCP · identity and permissions
1:40 – 1:50	**Break**
1:50 – 2:40	**Advanced techniques for project monitoring** Flow metrics, DORA, dashboards, anomaly detection, attention vs alerts
2:40 – 2:50	**Break**
2:50 – 4:00	**Hands-on: tool integration** Lab 3: repo → chat → LLM summary → flow dashboard

Client ToC modules covered

- Overview of Modern Project Management Tools
- Integrating Collaboration and Communication Tools
- Advanced Techniques for Project Monitoring
- Hands-On Exercise: Tool Integration

Software Factory thread

- Plain GitHub objects as the whole state; OpenSpec folders as the content — vendor-neutral and resumable
- The control path: event → caller stub → router → role → label moves
- Approvers named in config; GitHub's own notifications do the work

The tool landscape — choose by delivery model, not feature list

Work tracking

Jira + Atlassian Intelligence / Rovo · Azure DevOps + Copilot · GitHub Projects + Copilot · Linear · Asana AI · Monday · ClickUp Brain

Knowledge and specs

Confluence · Notion AI · SharePoint + Copilot · OpenSpec change folders · ADRs and design docs as reviewable artifacts

Collaboration

Microsoft Teams + Copilot · Slack AI · meeting recorders (Teams, Zoom, Fireflies, Otter) · email as the record

Monitoring and BI

Power BI · Looker · Grafana · Jira dashboards · DORA tooling (LinearB, Jellyfish, DX) · the Factory Console

AI assistants and agents

ChatGPT · Claude · Microsoft Copilot · Claude Code · GitHub Copilot coding agent · Cursor · orchestration (GitHub Actions, LangGraph)

Automation

Power Automate · n8n · Zapier · GitHub Actions · Model Context Protocol (MCP) servers that expose tools to assistants

Three questions before any tool: where does the system of record live, what must be integrated versus merely coexist, and who is the identity every automated action runs as?

How an event becomes an action: the control path

GitHub event

issue opened · label applied · comment 'Approved' · PR merged

Caller stub

~20 lines in the repo: triggers and a version pin, no logic

Router

Reads the event and the current label; names a role or replies why nothing ran

Role runs

One agent, one issue, a model chosen per stage

Visible trace

Label moved, comment posted, next actor @-mentioned. No silent green runs

Primitives every PM should know

- Webhook: the tool calls you when something happens
- API: you ask the tool
- Bot: an identity that acts in chat
- MCP: a standard way to give an AI assistant tools — your tracker, your docs, your calendar

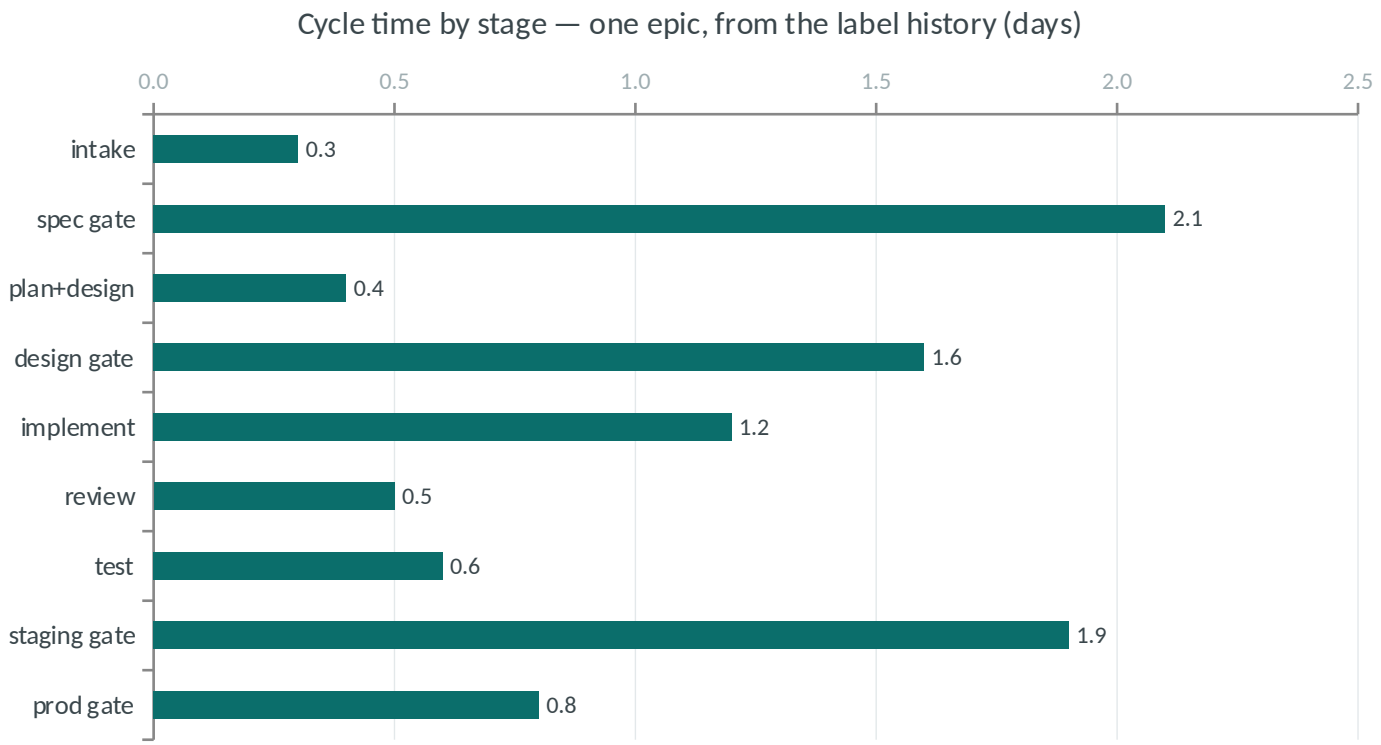
Identity and permissions

- Every automated action runs as someone
- Approvals are recorded in the tool that owns the decision
- In the factory, the Console posts approvals under the user's own GitHub identity — it adds zero new enforcement paths
- Approvers are a config file; the pipeline assigns and @-mentions them

Avoiding notification debt

- Teams / Slack for attention, email for the record, the assistant for questions
- One channel per project, one thread per epic
- Digest what is informational; interrupt only for decisions
- Say, on the object, what is expected of the next actor

Monitoring that answers questions: flow metrics and DORA



Where did the 9.4 days go? 6.4 of them were humans at gates. The agents took 3.0. Decision latency is the new constraint.

Flow metrics

- Throughput · cycle time · WIP · flow efficiency · ageing
- Read together: rising WIP with flat throughput is the earliest warning you get

DORA

- Deployment frequency · lead time · change-failure rate · time to restore
- In the factory: factory:deployed and factory:incident are the two labels that make these a query; the Ops role's smoke and soak checks feed them

Lab 3: wire the tools together

Repo → chat → LLM → dashboard

- In the sandbox GitHub organisation, connect a repository to a Teams / Slack channel
- Add an LLM summarisation step on issue and PR events (what changed, who is waiting, why)
- Build a small flow-metrics dashboard from the label history: cycle time by stage, WIP by state, blocked ageing
- Stretch: expose the tracker to an AI assistant through MCP and ask it delivery questions in plain English
- Stretch: add an anomaly rule — alert when blocked ageing exceeds two days

Deliverable A live integration from the system of record to the team's chat, with an LLM in the loop, and a dashboard that reads from the same events.

Closing round

Which one integration would you add to your current project first?

What did the LLM summary get wrong, and what would you give it to fix that?

Day 4 — Managing AI-driven delivery

0:00 – 0:45	**The future of project management** Managing people and agents · the PM as gatekeeper · skills that appreciate
0:45 – 1:45	**Best practices for project delivery in tech projects** Spec-driven · small batches · staging first · independent review · guardrails as controls
1:45 – 1:55	**Break**
1:55 – 2:55	**Practical guide to managing Gen AI projects** Non-determinism, evaluation, cost, privacy, responsible AI, model routing
2:55 – 3:05	**Break**
3:05 – 4:20	**Hands-on: run an epic through the factory** Lab 4: file it, plan the release, open G0 → G1 → G2 → GS, see what G3 asks

Client ToC modules covered

- The Future of Project Management
- Best Practices for Project Delivery in Tech Projects
- Practical Guide to Managing Gen AI Projects

Software Factory thread

- This is the day the factory is the subject, not the example
- Ten roles, the state machine, five gates — and why only GS and G3 are structural
- Guardrails: a hook, a deny list, a detector
- Model routing: spend quality where errors compound

From managing people to managing people and agents

What gets automated

- Coordination and hand-offs between stages
- Status collection and first-draft reporting
- First-draft artifacts: specs, plans, designs, test plans
- Implementation, review and testing of well-specified work
- Release assembly: dependency-ordered merges, staging checks

What stays human — and appreciates

- Scope, priorities and trade-offs
- Writing an unambiguous requirement
- Reading a design and a test report well enough to say no
- Opening the gates that put code on staging and in production
- Relationships, escalation, accountability



The pipeline waits on a human, not the other way round. When agents take minutes and approvals take days, decision latency becomes the delivery constraint. The PM's operating rhythm changes from weekly status to daily gates — and the PMO's dashboard changes from activity to attention.

Ten roles, five gates: the factory pipeline in one picture

#	Stage	Role	Produces	Gate	Who opens it
0	**Release**	`scrum`	Release plan for one milestone: scope, sequencing, risks, GO / NO-GO	G0	release scope approver — or the agent's own GO
1	**Intake**	`intake`	proposal.md + WHEN/THEN scenarios, opened as a PR; stops and asks if ambiguous	G1	spec approver reviews the PR, comments 'Approved'
2	**Plan**	`planner`	tasks.md — ≤ ~10 tasks of ≤ half a day, one PR each, mirrored into sub-issues		
3	**Design**	`architect`	design.md per repo: contracts, migrations, failure modes; reads the actual code first	G2	design approver — plan and design are one approval
4	**Dispatch**	`dispatch`	Marks tasks whose dependencies are merged as ready; no judgement, pure coordination		
5	**Implement**	`implementer`	Branch, tests, draft PR — exactly one task per session		
6	**Review**	`reviewer`	Independent session, different model; approve or send back		
7	**Test**	`qa`	Scenario → test mapping, suites green, test report on the PR		
8	**Deploy**	`release`	Dependency-ordered merges onto the epic branch, then integration, then promotion PRs	GS	staging approver: 'Approved' on the assembled epic
9	**Verify**	`ops`	Health and smoke checks, soak, archive the change, close the issue	G3	a human presses Merge on the promotion PR — never an agent

Beside the stages: a fast lane (factory:fast-track) for changes too small for the ceremony, and expedite (factory:expedite) for all of the ceremony and none of the pauses — which still stops at GS.

Delivery best practices, as the factory enforces them

Spec-driven delivery

A change is a proposal, acceptance scenarios, a design and a task list — each reviewed before the next is written.

Small batches

Tasks of at most half a day, one PR each, ordered so every intermediate merge is releasable: schema → service → consumer.

Staging first, always

Nothing reaches main that was not assembled and proven on an integration branch. Promotion PRs carry the staging evidence.

Independent review

A reviewer that never shares the implementer's context — in the factory, a different model, so the blind spots differ.

Guardrails as controls

A hook blocks the push, a deny list removes the merge tool, a detector files an incident. Not a policy document.

Release gating

A milestone is the release. Nothing starts until the whole set has been read, sequenced and approved — once.

Governance you can prove: controls, not instructions

1

Prevent

A pre-tool-use hook blocks any agent git push to main or master — every refspec form, force pushes, deletes. Tested like code.

2

Remove

A permission deny list removes the PR-merge tools from every agent session. Humans merge in the GitHub UI; that click is gate G3.

3

Detect

A detection workflow opens a factory:incident issue if a commit ever lands on main without a pull request, or by a PR that did not come from staging.

Plus: agent comments carry a marker so they never trigger themselves; a gate flip is checked against the approver list no matter who sends it.

What a PM takes from this

- A rule an AI system 'should follow' is a wish. A rule the toolchain enforces is a control.
- Ask of every AI initiative: what can it not do, and how do we know?
- Detection is the third layer, not the first — and it must page a person.
- Approver lists, gate evidence and decision logs are the audit trail regulators will ask for.
- The same three layers apply to a chatbot, an automation, or a coding agent.

Managing Gen AI projects: what is actually different

Non-deterministic output

The same input gives different answers. Pass / fail testing becomes evaluation on a curated set with a threshold.

Evaluation is the schedule

Most of the time goes into the eval loop, integration and change management — not the model.

Cost is tokens

Budget per run, per stage, per organisation. Route cheap models to volume, strong models to judgement.

Drift

Prompts, models and data all move. Version them; re-run the evals on every change.

Data in the prompt

Privacy and confidentiality apply to what goes in and what comes out. Client policy decides the assistant.

Vendor dependency

Keep state and content vendor-neutral; only the execution layer should be tied to one model provider.

The Gen AI project lifecycle



Responsible AI in one line each: transparency (say what the AI did), human oversight (a person holds the gate), auditability (the decision log), lawfulness (EU AI Act risk tiers, India's DPDP Act, the client's own AI policy).

Model routing: spend quality where errors compound

Stages	Preference chain	Why
scrum · intake · planner · architect · reviewer	`Fable 5 → Opus 5 → Sonnet 5`	A wrong release scope, spec, plan or design — or a missed review defect — multiplies downstream. The model delta is cheap by comparison.
implementer · qa · release	`Opus 5 → Sonnet 5`	Strong coding and testing on the volume stages, guard-railed by the merged design and the spec scenarios.
dispatch · ops	`Haiku 4.5 → Sonnet 5`	Mechanical coordination and checks. Fast and cheap; nothing here needs judgement.

Preference chains, not a single model

Before launching, the pipeline probes each model with a one-token ping and uses the first the credential can reach. A plan or tier gap degrades gracefully instead of failing the run.

Budgeting an estate

Cost per role per run × runs per epic × epics per release. The Console holds a token budget per organisation for its Q&A. Ask the same of any AI vendor: what does one unit of work cost?

The PM's version of this

Use the strong model for the artifacts people will sign off on; the fast one for the drafts, summaries and reminders. Review where the error would compound.

DAY 4 · HANDS-ON EXERCISE · 75 MINUTES

Lab 4: run an epic through the factory — in the approver seats

1 File

A requirement against a milestone in the sandbox estate. It parks in factory:backlog.

2 Plan release

Comment it on the tracker. Read the Scrum Master's scope, order, risks and verdict.

3 G0

Comment 'Approved'. Every backlog issue in the milestone enters intake at once.

4 G1

Review the proposal and WHEN/THEN scenarios. Send one back with questions — see factory:blocked work.

5 G2

Read tasks.md and design.md in one PR. Approve. The Dispatcher releases the unblocked tasks.

6 Watch

Implement → review → test run unattended. Read the review and the test report as they land.

7 GS

The epic is assembled and green. Approve; the Release Manager carries it to staging and verifies.

8 G3

See what the promotion PR asks of a human, and why no agent can press that button.

Roles rotate: release-scope approver, spec approver, design approver, tester. **Closing round:** *which of your current sign-offs are gates, and which are merely waiting?*

DAY 5 AGENDA · 4 HOURS

Day 5 — Capstone, governance and certification

0:00 – 0:45	**Case study discussion** The four-repository estate · porting the pipeline to another runner · what applies to you
0:45 – 0:55	**Break**
0:55 – 2:55	**Capstone project (teams of 3–4)** AI-enabled delivery plan for a case-study release, six deliverables
2:55 – 3:05	**Break**
3:05 – 3:40	**Capstone presentations and critique** 10 minutes per team, scored against the outcome measures
3:40 – 4:15	**Course conclusion, 30-60-90 plan, certification** Recap, scenario quiz, personal adoption plan, certificate

Client ToC modules covered

- Capstone Project & Case Study Discussion
- Course Conclusion and Certification

Software Factory thread

- The original deployment: four repositories, one pipeline — what it cost, what broke, what the gates caught
- The same shape on GitHub Copilot: what was vendor-neutral and what was not
- The capstone's governance design uses the factory's gate and guardrail vocabulary

Case study: one pipeline, four repositories

Repository	Stack	Role in the estate
`lighthouse-backend`	Python 3.12 · FastAPI · MySQL	Coordination repo — epics filed here
`lighthouse-ui`	React 19 · Vite · TypeScript	Frontend implementer
`lighthouse-sales`	FastAPI · Postgres/pgvector · Celery	Sales automation implementer
`lighthouse-whatsapp-server`	NestJS · Prisma · BullMQ	Messaging implementer

The problem it solved. An agent could write good code, but nothing could tell you where a piece of work was, what it was supposed to do, or who had agreed to it.

What each stage cost, in minutes and in tokens — and where the humans spent their time (mostly at G1, reading specs).

What broke. Cross-repo hand-offs without a shared token; runs that finished green having done nothing; sub-issues stranded with no state after a merge.

What the gates caught. A design that duplicated an existing module; a task ordering that would have shipped a consumer before its schema.

Second case: the same shape on GitHub Copilot

- State (issues, labels, milestones, PRs) and content (OpenSpec folders) are vendor-neutral — roughly 70% of the factory did not change
- What had to change: how an agent is defined and launched, and how permissions are enforced
- Role prompts ported almost verbatim: trigger → mission → numbered steps → guardrails
- Discussion for your context: a services engagement with a client PMO, a fixed-price project, a regulated environment — what carries over, what needs a different gate design?

DAY 5 · 2 HOURS IN TEAMS OF 3–4, THEN 10-MINUTE PRESENTATIONS

Capstone: an AI-enabled delivery plan for a real release

Brief: a release of 6–8 requirements for a case-study insurance policy-servicing application (teams may substitute a sanitised release of their own). Produce everything with AI; review everything as a team.

1. Release plan

Scope table, sequencing with the dependency that forces it, risks, what to drop, GO / NO-GO — in the Scrum Master's format.

2. Probabilistic forecast

P50 / P85 completion with the assumptions and the two scenarios that move it.

3. Risk register

AI-mined from the requirements and the case history; exposure, owners, early-warning signals.

4. Automation & reporting plan

Which reports and rituals are automated, from which sources, with what human review.

5. Governance design

Which gates exist, who holds them, which are delegated to an agent, which guardrails are controls.

6. Stakeholder pack

A sponsor-grade one-page status and a steering-committee narrative, generated from the plan.

Scored against the six business outcomes: predictability, reporting, risk, automation, governance, AI readiness. Peer critique first, trainer critique second.

Leaving with a plan, not just a certificate

30 days

One report to automate

- Pick the report that costs the most hours
- Build it from the system of record with an LLM narrator
- Run it in parallel with the manual one for two cycles

60 days

One forecast, one gate

- Replace a committed date with a P50 / P85
- Redesign one sign-off as a real gate with evidence and an approver list
- Put an attention queue in front of the sponsor

90 days

One AI-driven initiative

- Govern a Gen AI project with evals, budgets and guardrails as controls
- Or: factory-enable one sandbox repository and run a first epic
- Report what changed in predictability and reporting time

Assessment and certificate

- Capstone 60% + scenario-based quiz 40%
- Certificate of completion issued on passing; recording and materials retained by the client

Take-home resources

- The Software Factory handbook and case studies (open source, MIT)
- Prompt library, forecast templates, automation templates, reading list

Lab setup and system requirements

Hardware (per participant)

- Laptop with 8 GB RAM or more
- Chrome or Edge, stable internet
- Microphone and camera
- No GPU; nothing heavy runs locally

Software and accounts

- GitHub account (free), added to the sandbox organisation
- A Generative AI assistant: Claude, ChatGPT or Microsoft Copilot — the client's enterprise assistant preferred, to respect data policy
- Excel or Google Sheets
- Teams or Slack sandbox channel
- Optional stretch: VS Code, Node.js 20+, Python 3.11+

Provided by the trainer

- Sandbox GitHub organisation with the Software Factory installed on a case-study repository
- Approver lists configured for the batch
- Factory Console deployed for participants
- LLM API credits for the factory runs on Days 4–5
- Datasets, prompt library, lab guides, templates

Lab cost per participant covers the sandbox environment, the agent-run credits for the labs and capstone, and AI-assistant usage where the client does not supply its own. All required software is available from our side; the client supplies only its own enterprise AI assistant if it prefers one.

Delivery format, and topics we can add

Mode	Online, instructor-led, recorded — recording provided to the client
Duration	20 hours: 5 half-days × 4 hours (4.5 with extended labs)
Batch size	12–20 participants; capstone in teams of 3–4
Materials	Slide deck, lab guides, prompt library, forecast and automation templates, sandbox estate, the Software Factory handbook
Assessment	Capstone 60% + scenario quiz 40% → certificate of completion
Language	English

Beyond the initial curriculum (60–90 min each, or a sixth half-day)

- Agile estimation and planning with AI: story splitting, backlog grooming, sprint-planning assistants
- Vendor and contract management for AI-delivered work: acceptance when an agent wrote the code, IP of generated artifacts
- Change management and role redesign: BAs, testers and tech leads on an agent-driven pipeline
- Portfolio and PMO-level AI: cross-project forecasting, capacity planning, multi-repository estates
- Agentic AI deep-dive: event-driven routers versus graph engines, MCP, evaluating agent runs
- Build the client's own factory: factory-enable one sandbox repository and run a first epic

A plain requirement goes in.
A governed, proven change comes out.
The PM holds the gates.

That is the delivery model this program prepares Project and Delivery Managers to run — and the one they practise, hands-on, for 20 hours.

`factory:deployed`

Jerry Kurian · GenAI People
jerry@genaipeople.com · genaipeople.com · github.com/genai-jerry/claude-software-factory